

Конструктори и Деструктори

Конструктор

- Специална член-функция на класа, без тип на връщане
- Извиква се автоматично при създаване на обект
- Има същото име като класа
- Инициализира данните на обекта и гарантира валидно начално състояние

Конструктор с параметри

```
Student(const string& -name, double -grade)
    : name(-name), grade(-grade)
{
    проверки за валирност
}
```

← прави инициализирането в `initiation` листа, спестява създаването на празно `name` и `grade`

Инициализиращ списък е задължителен при:

- константна променлива
- класове без `default` конструктор
- член-данни от тип референция

Редът на инициализацията зависи от подредбата на член-данните, а не от инициализиращия списък.

При дефиниция на друг конструктор, `default` не се генерира автоматично.

Деструктор

- Извиква се автоматично при унищожаване на обекта
- Освобождава ресурси и името му е: `~ClassName()`
- Извиква се при:
 - край на `scope`
 - `delete` или `delete[]`
 - унищожаване на временни обекти
- В него не трябва да се хвърля `exceptions`
- При динамична памет той трябва да извика `delete` или `delete[]`.

- Редът на унищожаване на обекти е обратен на този при създаване

Указател this

- Указател към текущия обект
- Използваме го при връщане на текущия обект

Конвертиращ конструктор

- Конструктор с един параметър, позволява ~~на~~ преобразуване от друг тип към типа на класа.

Explicit конструктори

- explicit - по подразбиране се използва
- implicit - само когато преобразуването е естествено и няма риск от грешка

Масиви от обекти - поредица от инстанции на клас

- За всеки елемент се извиква конструктор при създаване и деструктор при унищожаване.